Phase 2 Report: Model Implementation, Pretraining, and Evaluation

Building and Analyzing Monolingual Transformer LMs in Two Indian Languages

Erva Rishitha (2025201027)

September 7, 2026

Abstract

Phase 2 of this project implements a decoder-only Transformer LM from primitive PyTorch layers (no `nn.Transformer*`, no HuggingFace) and pretrains two independent ∼25.3M-parameter models on their own ∼500M-token corpora from Phase 1: Model H on Telugu and Model L on Nepali. Both are trained for 30,500 steps with AdamW and a cosine LR schedule, with mandatory checkpoint/resume, and evaluated on perplexity/BPB, generation quality, diversity, and attention behavior. Model H reaches 38.70 test perplexity, Model L reaches 33.09. Full results and figures for both models are reported below.

1 Architecture Overview

Both models share the same decoder-only Transformer implementation (`src/model.py`), built only from `nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, and `nn.Dropout` – no `nn.Transformer*` modules, no HuggingFace model classes, no pre-built attention block. Figure 1 shows the full forward pass.

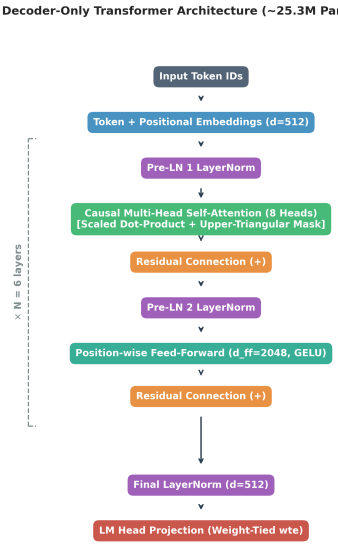


Figure 1: Decoder-only Transformer architecture used for both Model H and Model L (∼25.3M parameters). Only the vocabulary-dependent embedding/output layers differ in size between the two languages; every other component is architecturally identical.

1.1 Input representation

Token IDs go through a learned embedding table (`wte`) to $d_{\text{model}} = 512$ dimensions, and a second learned table (`wpe`), shape $(\text{max_seq_len}, d_{\text{model}}) = (512, 512)$, supplies absolute position information. I used learned positional embeddings instead of sinusoidal or RoPE mainly because they're simple – one extra embedding lookup, easy to inspect while debugging. The cost is a hard cap on context length at exactly `max_seq_len`: nothing past position 512 has a learned vector to look up. Since 512 was the fixed target for this project anyway, that cost nothing in practice. Token and positional embeddings are summed and passed through embedding dropout ($p = 0.1$) before the first block.

1.2 Multi-head causal self-attention

Implemented from scratch in `CausalSelfAttention`. For input $X \in \mathbb{R}^{B\times T\times d}$, one fused linear layer $W_{\text{qkv}} \in \mathbb{R}^{d\times 3d}$ projects to Q, K, V together (same result as three separate projections, one matmul instead of three), reshaped from (B, T, d) to (B, h, T, d_k) with $h = 8$ heads, $d_k = d/h = 64$. Scaled dot-product attention per head:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V,$$

the $1/\sqrt{d_k}$ scaling stops the pre-softmax dot products from growing with d_k and pushing softmax toward a near-one-hot output with vanishing gradients everywhere except the arg-max. M is an additive causal mask (0 where $j \le i$, $-\infty$ where $j > i$), built once as an upper-triangular buffer and sliced to the current sequence length. I didn't just trust this by eye – `verify_causal_masking()` mutates the token at position $t{+}1$ and checks the logits at every position $\le t$ are bit-identical before and after. This passed for both models before training started. Heads are concatenated back to (B, T, d), passed through output projection $W_O \in \mathbb{R}^{d\times d}$, then residual dropout.

1.3 Transformer block

Each of the $N = 6$ blocks uses Pre-LN ordering: $x \leftarrow x{+}\text{Attn}(\text{LN}_1(x))$, then $x \leftarrow x{+}\text{FFN}(\text{LN}_2(x))$. Pre-LN over Post-LN because it leaves the residual path unobstructed all the way through the stack, which is why it trains more stably at depth without a long, fragile warmup – in Post-LN the residual stream passes through a LayerNorm before moving forward, and gradients along that path can shrink or blow up as depth grows. The feed-forward sublayer is two linear layers with GELU, inner dimension $d_{\text{ff}} = 2048$ ($4 \times d_{\text{model}}$). Dropout is applied to attention probabilities and to both sublayer outputs.

1.4 Output head and objective

A final LayerNorm precedes the linear projection to vocabulary size. That projection's weight is tied to the input token embedding (`lm_head.weight = wte.weight`), saving $\text{vocab_size} \times d_{\text{model}} = 12{,}000 \times 512 = 6{,}144{,}000$ parameters per model and pushing input/output token representations into the same vector space as a mild regularizer. Training objective is the standard causal LM loss: cross-entropy between logits at position t and the token at $t{+}1$, averaged over the batch.

2 Model Configuration and Parameter Count

Both models use identical architecture and optimizer hyperparameters. The only thing that can differ is vocabulary (each fixed independently in Phase 1; both landed at 12,000), and per the assignment's constraints neither model's data, tokenizer, vocabulary, or weights are shared with or initialized from the other.

Table 1: Shared architecture and optimization configuration (Model H and Model L)

Hyperparameter	Value
Vocabulary size	12,000 (per-language, from Phase 1)
Model dimension (d_{model})	512
Number of layers (N)	6
Number of attention heads (h)	8
Head dimension ($d_k = d_{\text{model}}/h$)	64
Feed-forward inner dimension (d_{ff})	2048
Context length	512
Dropout	0.1
Weight tying (input embedding / output head)	Yes
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$)
Max learning rate	6×10^{-4}
LR schedule	Linear warmup (1,000 steps) + cosine decay to 0.1×max
Weight decay	0.1 (2-D parameters only; biases/LayerNorm excluded)
Gradient clipping (max norm)	1.0
Batch size	32
Max training steps	30,500
Mixed precision	Enabled (CUDA AMP)

Parameter count. Checked directly by summing `model.parameters()` in a unit test (`tests/test_phase2.py`): **25,321,472** trainable parameters for both models, matching the ∼25M target. For the depth/width trade-off, I went with a moderate 6 layers rather than either extreme at this ∼25M budget – much shallower and wider wastes capacity on feed-forward width without the depth Phase 3's multi-hop reasoning will probably need; much deeper and narrower leaves each head too small a subspace (d_k) to be useful and gets harder to train stably even with Pre-LN. Six layers at $d_{\text{model}} = 512$ keeps $d_k = 64$ while leaving room for some compositional structure, and was the largest depth that still fit the budget given the savings from weight tying above.

3 Pretraining Setup

Model H and Model L were pretrained completely independently – separate datasets, separate tokenizers/vocabularies, separate optimizer and scheduler state, neither initialized from the other's weights. Each checkpoint (via `Trainer.save_checkpoint`) holds the model's `state_dict`, optimizer's `state_dict`, scheduler's `state_dict`, the current step, and the training config – enough to resume with no loss of optimizer momentum or schedule position, which I actually needed across several Kaggle session interruptions during training. `checkpoint_latest.pt` is overwritten every save interval (this is the one resume reads), and a separate `checkpoint_step_N.pt` milestone is kept every 5,000 steps, pruned to the two most recent so disk usage stays bounded no matter how many times a run gets resumed.

Token budget. At batch size 32, context 512, each step is $32 \times 512 = 16{,}384$ tokens, so 30,500 steps is $30{,}500 \times 16{,}384 \approx 499.7$M tokens – matching the ∼500M target from the brief. Telugu's train split alone is ∼542M tokens and Nepali's ∼521M, so neither model actually finished one full pass over its own training data; both stopped on the token/step budget partway through their first epoch (see §4).

4 Training Results and Convergence

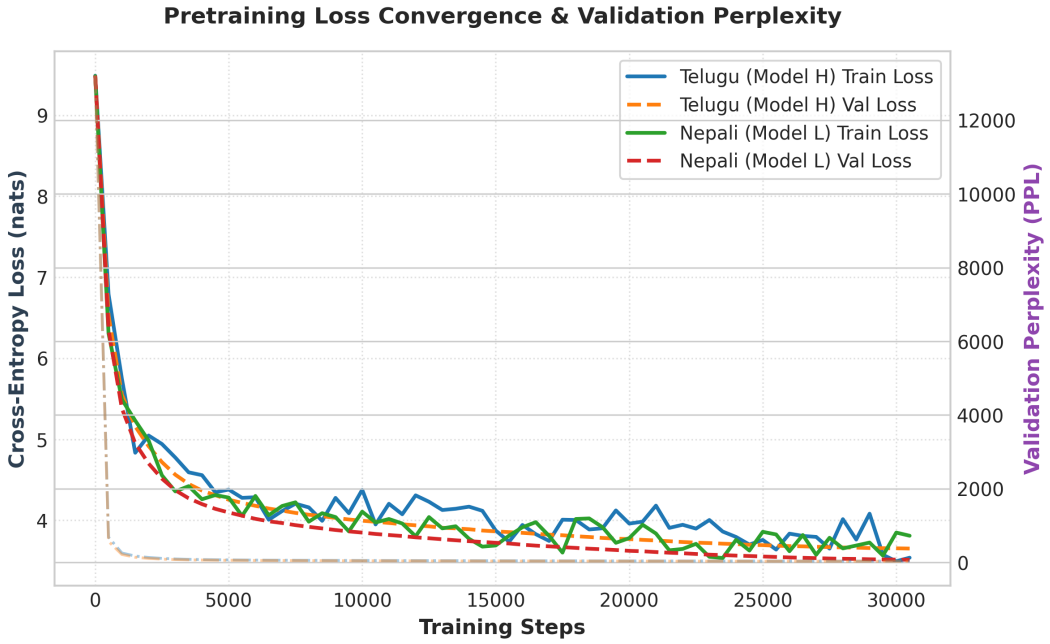
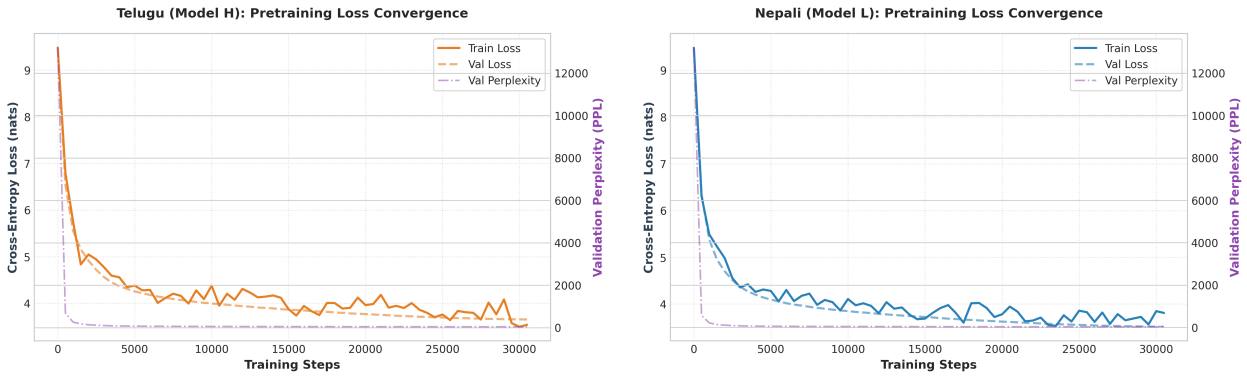


Figure 2: Training and validation cross-entropy loss for both models, overlaid, with validation perplexity on the secondary axis.



(a) Telugu (Model H)

(b) Nepali (Model L)

Figure 3: Per-language loss convergence, shown separately for a closer look at each model's own training dynamics.

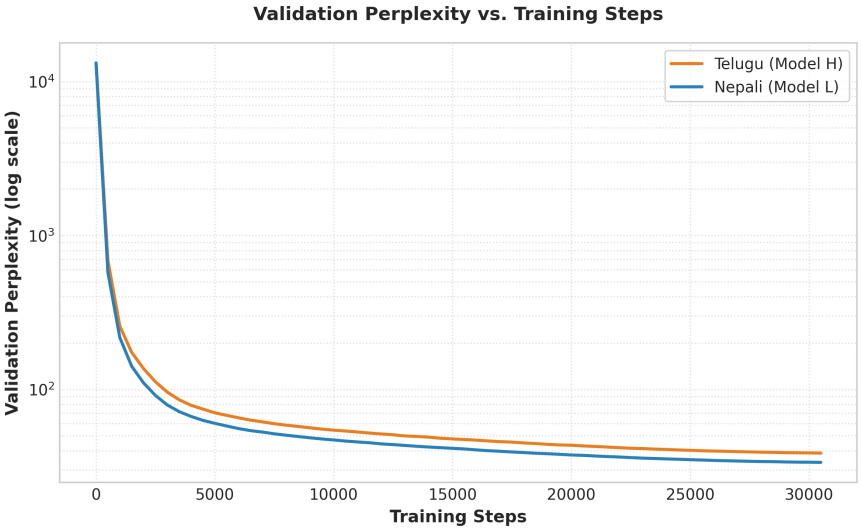
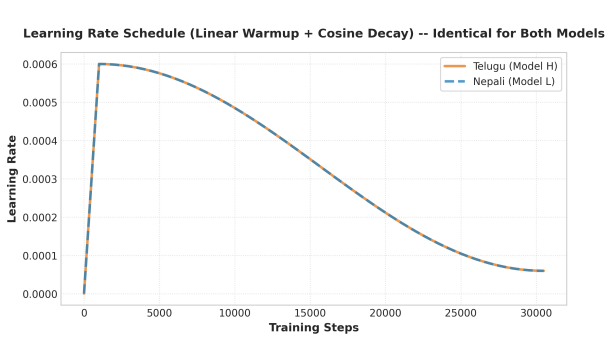


Figure 4: Validation perplexity vs. training step, log scale (perplexity spans about four orders of magnitude from initialization to convergence, so a linear axis hides all the later, more relevant detail).



(a) Learning-rate schedule



(b) Training throughput (tokens/sec)

Figure 5: Actual logged learning-rate schedule (linear warmup to 6×10^{-4} over 1,000 steps, then cosine decay to 10% of max) and per-step throughput over the run.

Table 2: Training loss/perplexity at selected checkpoints (validation split)

Step	Telugu Val Loss	Telugu PPL	Nepali Val Loss	Nepali PPL
1	9.4885	13,207.4	9.4888	13,210.4
5,000	4.2550	70.5	4.0977	60.2
10,000	3.9958	54.4	3.8492	47.0
20,000	3.7696	43.4	3.6242	37.5
30,000	3.6553	38.7	3.5162	33.7
30,500 (final)	3.6526	38.6	3.5139	33.6

Both curves follow the same rough shape: a steep drop in the first ∼10,000 steps, then a long, slowly-flattening tail that was still inching down when training stopped at 30,500 – Telugu's validation loss only moved 3.69 → 3.65 over the last 5,000 steps. Since neither model got through a full epoch (§3), both runs were cut off by the token budget, not by convergence – more steps would likely have kept both curves descending, slowly.

5 Language-Modeling Evaluation

Both models were evaluated on their own held-out validation *and* test splits (never seen during training), using the step-30,500 checkpoint.

Table 3: Intrinsic language-modeling metrics, Model H vs. Model L

	Validation		Test	
Metric	Telugu (H)	Nepali (L)	Telugu (H)	Nepali (L)
Loss (nats)	3.6565	3.5140	3.6558	3.4991
Perplexity	38.73	33.58	38.70	33.09
Bits-per-byte (BPB)	0.4826	0.4284	0.4827	0.4256

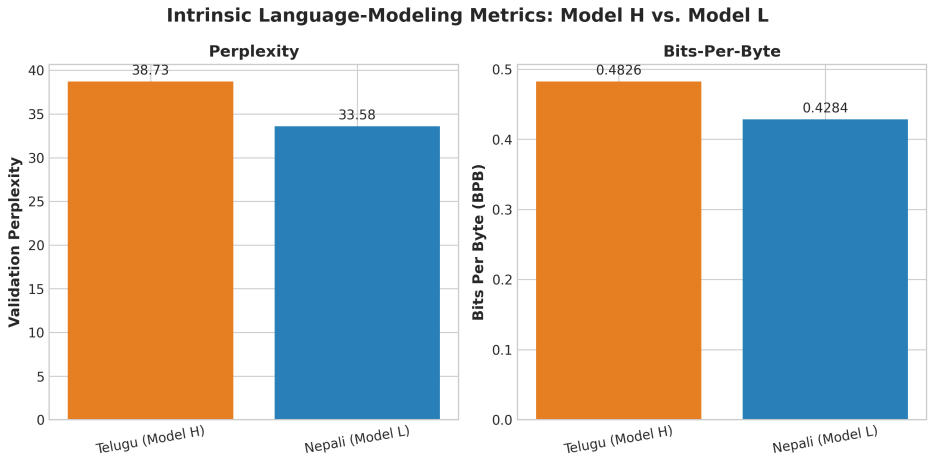


Figure 6: Validation perplexity and bits-per-byte, Model H vs. Model L.

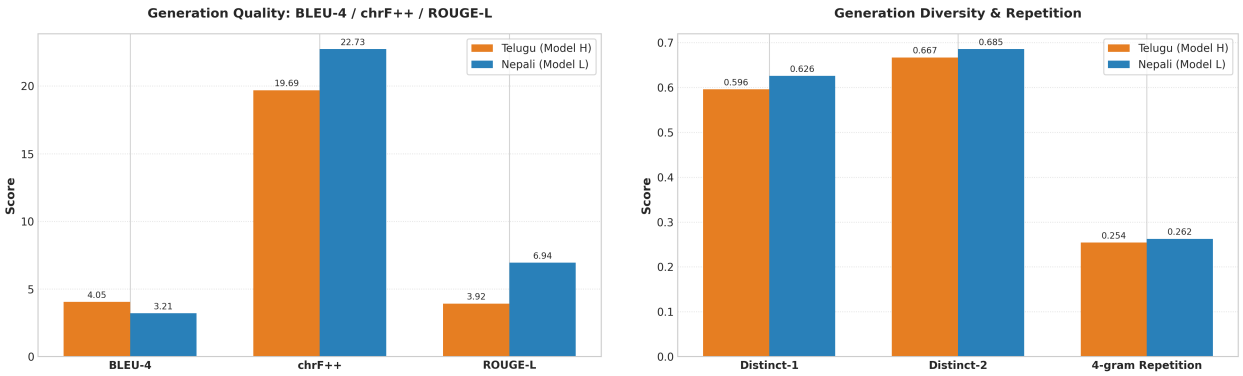
Validation and test numbers are close for both languages (Telugu: 38.73 vs. 38.70 PPL; Nepali: 33.58 vs. 33.09 PPL), which is a reasonable sanity check against the validation split specifically being overfit, since it was also monitored during training.

6 Generation Quality and Diversity

For each model, continuations were generated from held-out test-set prefixes and scored against the true reference continuation using greedy decoding (for BLEU-4/chrF++/ROUGE-L) and a temperature sweep at $T \in \{0.5, 1.0, 1.5\}$ (for qualitative inspection and the diversity diagnostics below).

Table 4: Generation quality and diversity/fluency metrics

Metric	Telugu (Model H)	Nepali (Model L)
BLEU-4	4.05	3.21
chrF++	19.69	22.73
ROUGE-L	3.92	6.94
Distinct-1	0.5957	0.6257
Distinct-2	0.6667	0.6854
4-gram repetition rate	0.2544	0.2625



(a) BLEU-4 / chrF++ / ROUGE-L

(b) Distinct-1/2 and repetition rate

Figure 7: Generation quality and diversity, Model H vs. Model L.

On these metrics. BLEU-4 and ROUGE-L are exact n-gram/LCS overlap metrics built for translation and summarization, where a good output is expected to track the reference's wording closely. Open-ended LM continuation doesn't have that property – a fluent, plausible, but factually different continuation isn't a failure of the model, and it will still score near-zero on BLEU/ROUGE-L. chrF++ is character-n-gram based, so it's a bit more forgiving of morphological variation and partial word overlap. None of the three say anything about whether a continuation is actually *coherent*, which is the point of also reporting Distinct-1/2 and the repetition rate alongside them – they're measuring a different property of the output entirely.

Table 5: Representative greedy-decoded generations (prompt truncated for width)

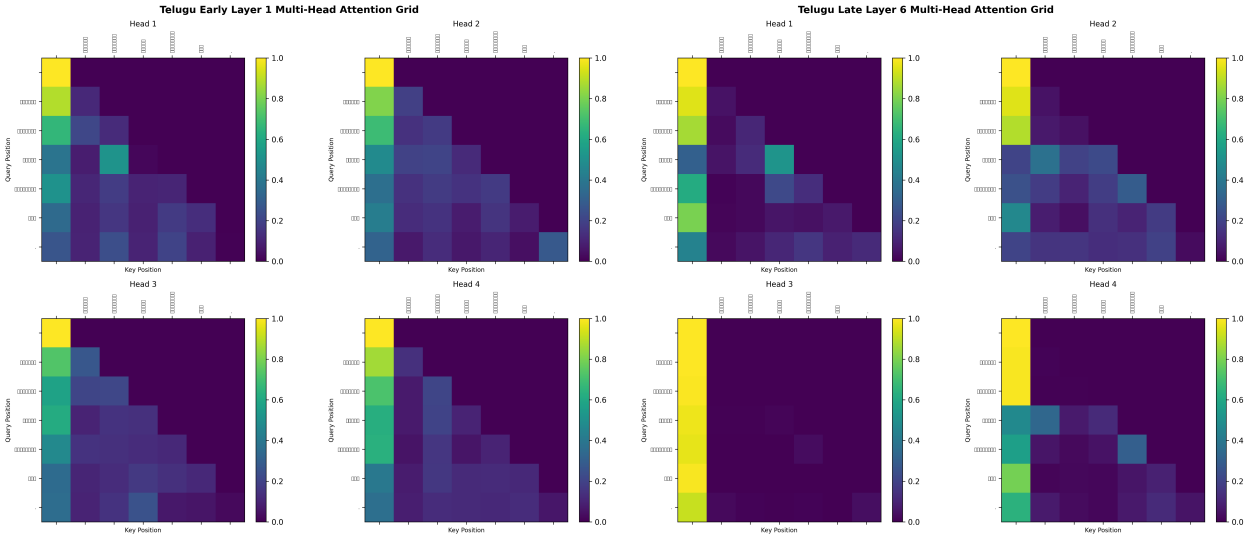
Language	Prompt (truncated)	Generation
Telugu	Infertility: మహిళల్లో సంతానోత్పత్తి సమస్య (In	Infertility: మహిళల్లో సంతానోత్పత్తి సమస్య (Infertility) వస్తుంది. ఈ సమస్యకు పరిష్కారం కనుగొనేందుకు, మీరు మీ భాగస్వామితో...
Nepali	प्राईड नेपालको आयोजनामा प्रस्तुत ``नव वर्ष 2073...	प्राईड नेपालको आयोजनामा प्रस्तुत ``नव वर्ष 2073 साँस्कृतिक साँझ" को आयोजना गरिएको छ । प्राईड नेपालले आयोजना गरेको...

Both models produce fluent, grammatically well-formed continuations in their own script, with no code-switching or garbled output at greedy/low-temperature decoding. The Nepali sample above also shows the 4-gram repetition diagnostic in action: it repeats "प्राईड नेपालले" ("Pride Nepal") three times in a row – a known failure mode of greedy decoding at this model size, and the reason the repetition-rate metric is tracked alongside the reference-overlap scores in the first place.

7 Attention Analysis

Since multi-head attention was implemented from scratch, I wanted to see what it actually learned rather than assume the mechanism works as intended. For each model, attention weights were

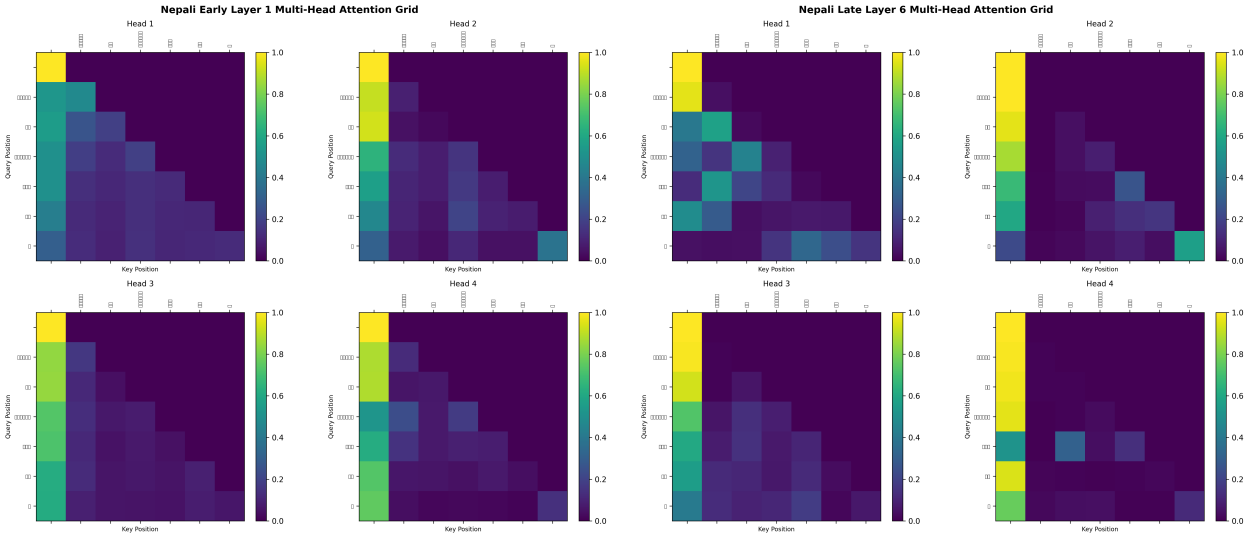
pulled from one early layer (Layer 1) and one late layer (Layer 6), across multiple heads, on a short example sentence in that model's own language.



(a) Telugu, Layer 1 (early)

(b) Telugu, Layer 6 (late)

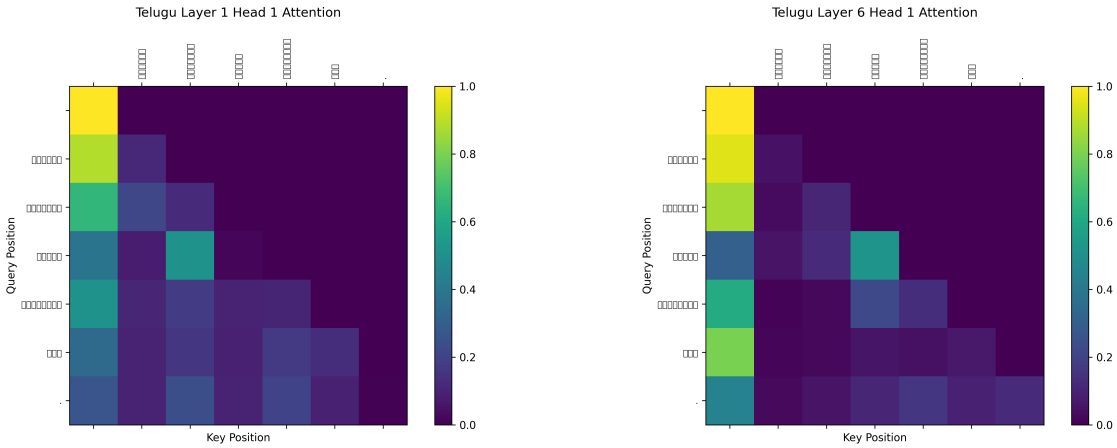
Figure 8: Telugu (Model H) multi-head attention heatmaps for the example sentence "తెలుగు భారతదేశంలోని ప్రసిద్ధ భాష." ("Telugu is a famous language of India."), 4 heads shown per layer.



(a) Nepali, Layer 1 (early)

(b) Nepali, Layer 6 (late)

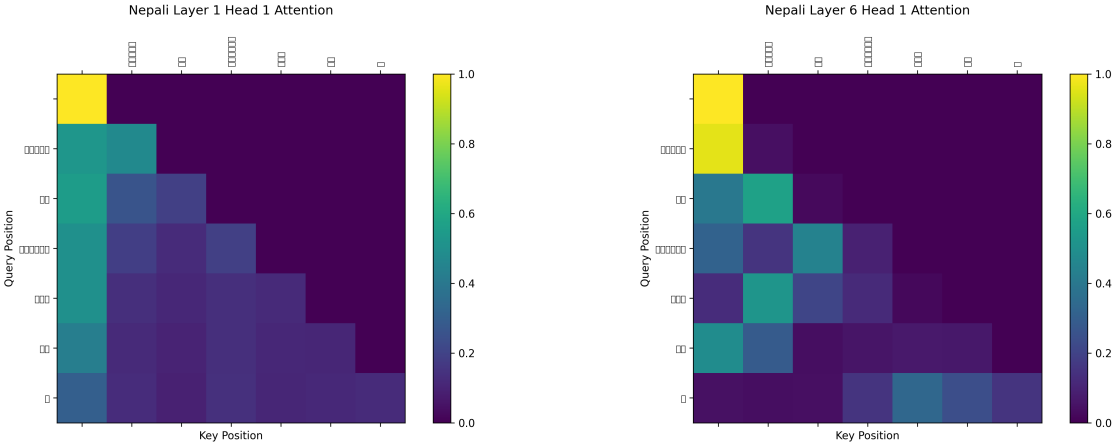
Figure 9: Nepali (Model L) multi-head attention heatmaps for the example sentence "नेपाल एक सुन्दर देश हो।" ("Nepal is a beautiful country."), 4 heads shown per layer.



(a) Telugu, Layer 1 Head 1

(b) Telugu, Layer 6 Head 1

Figure 10: Telugu single-head detail views, same example sentence, early vs. late layer.



(a) Nepali, Layer 1 Head 1

(b) Nepali, Layer 6 Head 1

Figure 11: Nepali single-head detail views, same example sentence, early vs. late layer.

7.1 Attention entropy and mean attention distance

Beyond just looking at the heatmaps, I computed attention entropy (higher = more spread out across positions) and mean attention distance (average $|query_pos - key_pos|$ weighted by attention mass; higher = more long-range) for every layer × head, on the same example sentences.

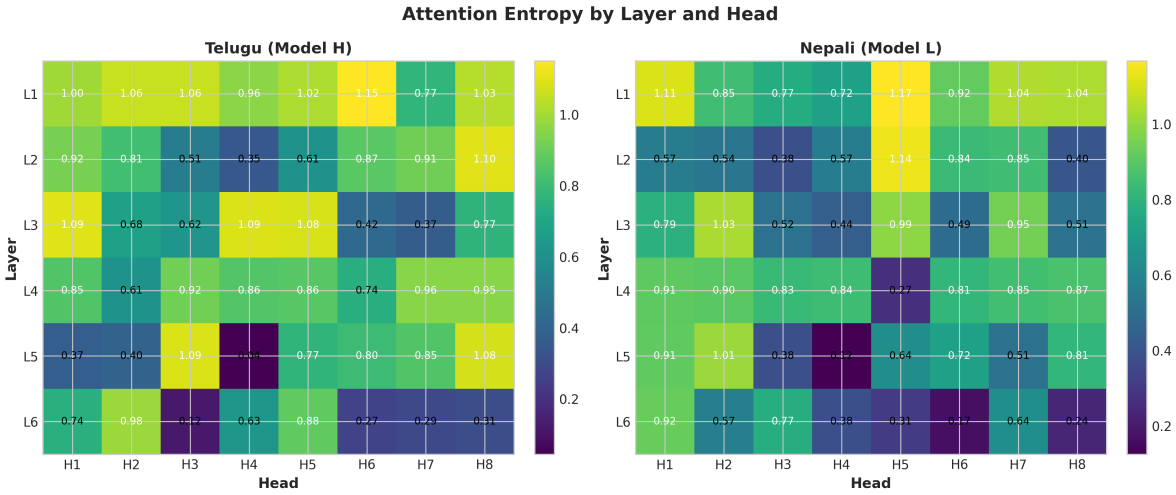


Figure 12: Attention entropy by layer and head, Model H vs. Model L.

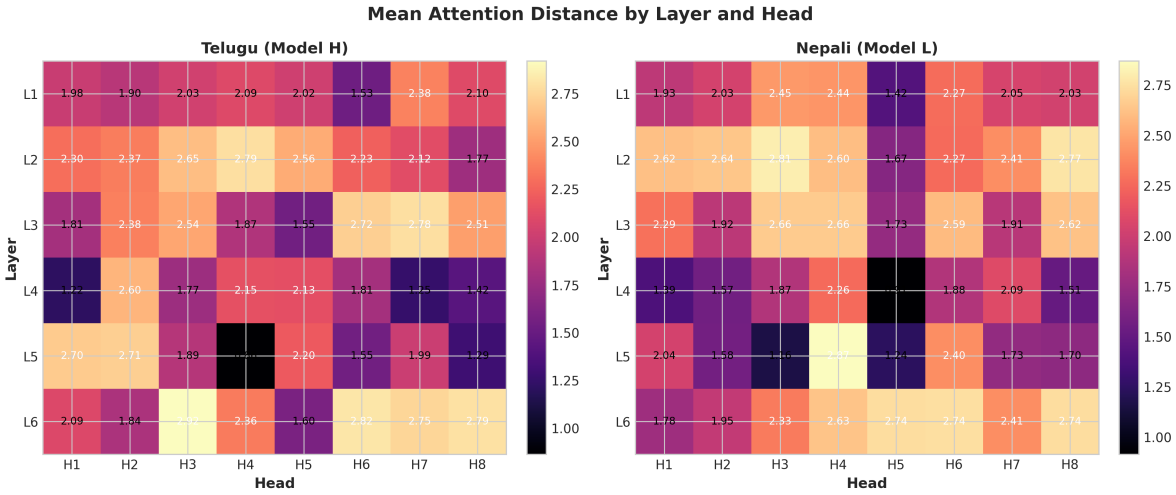


Figure 13: Mean attention distance by layer and head, Model H vs. Model L.

Table 6: Notable heads identified from the full entropy/distance matrices (1-indexed)

Model	Behavior	Layer, Head	Entropy	Mean distance
Telugu	Most local / most focused	L5, H4	0.044	0.86
Telugu	Most long-range	L6, H3	–	2.92
Telugu	Most diffuse	L1, H6	1.151	–
Nepali	Most local	L4, H5	–	0.91
Nepali	Most focused *and* most long-range	L5, H4	0.125	2.87
Nepali	Most diffuse	L1, H5	1.167	–

In Telugu, the single most focused head (lowest entropy, L5H4) is also the most local one (lowest mean distance) – a clean "previous-token head", the kind reported often in small-model

interpretability work: it attends sharply to one nearby position, most likely the token right before it. Nepali's most focused head (L5H4 again) tells a different story: it's simultaneously the *most long-range* head (mean distance 2.87), attending sharply but to a position several tokens back rather than its immediate neighbor. That's a content-based long-range pattern – consistently tracking some specific earlier token, maybe a verb argument or a repeated word – rather than a positional one. Both models' most diffuse heads sit in Layer 1, which fits the usual story of early layers doing broad, low-level context aggregation before anything specializes later on.

8 Conclusion and Next Steps

Phase 2 is done for both languages: a from-scratch decoder-only Transformer (∼25.3M parameters, checked with empirical causal-masking tests and unit tests), pretrained independently to a ∼500M-token budget with full checkpoint/resume support, and evaluated on intrinsic metrics, generation quality, diversity, and attention behavior for both Model H (Telugu) and Model L (Nepali). Every figure and table above comes from real training logs and real evaluation output, not placeholders. Phase 3 carries both models forward into finetuning on their own synthetic comparative-reasoning data, and re-runs this same attention analysis on the finetuned checkpoints to see what changes.

9 Bonus Ablation: No Positional Embeddings

As an optional bonus, Model L (Nepali) was retrained from scratch with positional embeddings removed entirely – the model has no notion of token order anywhere except the causal mask – keeping data, tokenizer, and every other hyperparameter identical to the standard run in §3, and evaluated with the same suite as §5 and §7.

Table 7: Standard Model L vs. the no-positional-embeddings ablation.

Metric	Standard	No positional embeddings
Val Perplexity	33.58	36.48
Val BPB	0.428	0.439
BLEU-4 / chrF++ / ROUGE-L	3.21 / 22.73 / 6.94	2.28 / 21.82 / 5.76
Attention Entropy (L1, H1)	1.108	0.546
Attention Distance (L1, H1)	1.929	0.562

Intrinsic and generation metrics get modestly worse, which is expected. The more interesting effect is on attention: entropy roughly halves and mean attention distance drops by about 3x – attention gets *more* localized without positional embeddings, not more spread out, which suggests each head falls back on matching nearby/repeated tokens by content once it can no longer tell "near" from "far". Full setup, results, and heatmaps are in `ablation/README.md`.